

Lab 4 – Resource Protection

Requirements

1. Design an RT application which:
 - a. Solves the Dining Philosophers problem for 8 philosophers
 - b. Provides a module (.h and .c files) which protects access to the UART with a binary semaphore and allows text strings to be printed to the UART through a module-defined function
 - c. Each philosopher must print out their state (Thinking, Hungry, or Eating) every time they change state
 - d. Philosophers are to remain in the Thinking and Eating states for random amount of time that is not less than 1 and not more than 5 seconds
2. Your solution must be composed of multiple tasks
3. Your solution must use binary semaphore(s) to protect variables that are shared between tasks and to protect the UART
4. You must demonstrate proper operation of your board during the lab time
5. You must submit your source code (only the source code, and only the files that you modified or created).
6. You must submit a design description for your solution. This can be a simple document with diagrams and text explaining how your program works and how it meets the requirements above (PDF preferred).

Hints

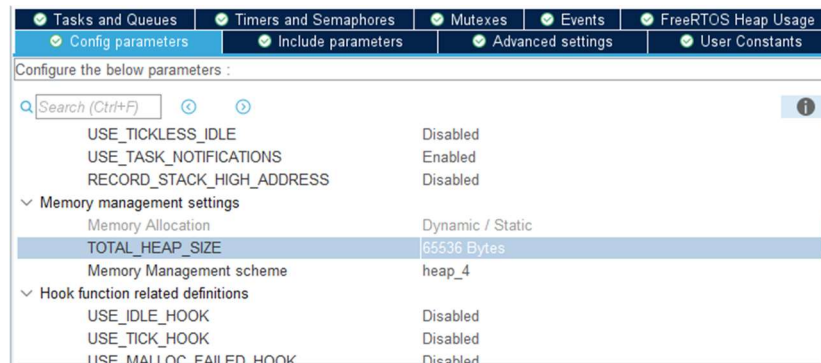
- Design and implement the “print” module first, test that module with a simple program (perhaps your solution from Lab 2)
- You can use the `rand()` function from the `stdlib.h` library – you should consider writing a function, which uses `rand()`, and returns a random number that is constrained to between 1 and 5 seconds (i.e., 1000msec and 5000msec)
- You will almost certainly need to increase the stack size from the default of **256*4** – I have the value set to **1024*4**.
- Remember to change the initial value of the binary semaphore(s) in the CubeMX generated code from zero to one.
- Your Philosopher tasks will need some way of identifying themselves (basically, they need to know where they are seated at the table). A way to do that is to pass a value into each task, for example:
 - Declare a global variable for each task:

```
int Philosopher1 = 1;
```

- Pass that value into the task:

```
Philosopher1_Handle = osThreadNew(PhilosopherTask,
                                   &Philosopher1,
                                   &Philosopher1_attributes);
```

- You cannot pass a literal into the task, the argument to the task is a point, that also means that you must have distinct variables that you pass in to each task
- You will also have to change the heap size in CubeMX, set it to 65536. “Middleware...FreeRTOS” then “Config Parameters” tab and set TOTAL_HEAP_SIZE to 65536.



Assessment Criteria

- Multiple Task solution
 - 1pts
- Data Protection
 - 1pts
- Resource (UART) Protection Module
 - 2pts
- Solution to Dining Philosophers problem
 - 4pts
- 2pts – Code quality (readability, comments, structure, etc.)
- 3pts deducted for each day that the demo is late
- Submit your source code – you must submit all source code that you modified or created. Do not submit the entire project directory, just the generated files that you modified and any additional files that you created.

Objective

The purpose of this lab is to get you familiar with creating a reusable module (for protected printing) and protecting a resource (UART) so any task can use it using a binary semaphore.